

RTL Design Sherpa

APB SMBus Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Smbus Mas Index....8

2 APB SMBus - Overview....9

2.1 Overview....9

2.1.1 Key Features....9

2.1.2 Applications....10

2.1.3 Block Diagram....10

2.2 Architecture....10

2.2.1 Module decomposition....10

2.2.2 The open-drain contract....13

2.2.3 Bit timing, fast mode and clock stretching....13

2.2.4 Timeout....15

2.2.5 Bus recovery....16

2.2.6 Busy, completion and the abort discipline....18

2.2.7 PEC....19

2.2.8 Interrupts....19

2.2.9 Soft reset, FIFO reset and the two-cycle strobe....20

2.2.10 Clock domains (CDC_ENABLE)....20

2.3 Transaction Table....21

2.3.1 Register Summary....22

2.4 Parameters....23

2.4.1 Integration notes....24

2.5 Waveforms....25

2.5.1 Waveform 1.1: Byte Write (Start + Address)....25
2.5.2 Waveform 1.2: Byte Read....25
2.5.3 Waveform 1.3: Clock Stretching....25
2.5.4 Waveform 1.4: Multi-Master Arbitration....26
2.5.5 Waveform 1.5: Packet Error Check (PEC)....26
2.5.6 Waveform 1.6: Timeout Abort with Bus Recovery....27
2.6 Target mode....27
2.6.1 The target PEC never counts bytes....28
2.6.2 Stretching, or not....28
2.6.3 One engine on the wire....28
2.7 Limitations....28
2.8 Navigation....29
3 APB SMBus - Register Map....30
3.1 Overview....30
3.1.1 Register Summary....30
3.2 Functional Description....31
3.2.1 SMBUS_CONTROL (0x00)....32
3.2.2 SMBUS_STATUS (0x04)....33
3.2.3 SMBUS_COMMAND (0x08)....38
3.2.4 SMBUS_SLAVE_ADDR (0x0C)....40
3.2.5 SMBUS_DATA (0x10)....41
3.2.6 SMBUS_TX_FIFO (0x14)....41
3.2.7 SMBUS_RX_FIFO (0x18)....42
3.2.8 SMBUS_FIFO_STATUS (0x1C)....42

3.2.9 SMBUS_CLK_DIV (0x20)..... 42
3.2.10 SMBUS_TIMEOUT (0x24)..... 43
3.2.11 SMBUS_OWN_ADDR (0x28)..... 44
3.2.12 SMBUS_INT_ENABLE (0x2C)..... 44
3.2.13 SMBUS_INT_STATUS (0x30)..... 46
3.2.14 SMBUS_PEC (0x34)..... 47
3.2.15 SMBUS_BLOCK_COUNT (0x38)..... 48
3.2.16 SMBUS_SLAVE_CTRL (0x3C)..... 49
3.2.17 SMBUS_SLAVE_STATUS (0x40)..... 50
3.3 The FIFO Contract..... 51
3.4 Programming Sequences..... 52
3.5 Remaining limitations..... 53
3.6 History..... 54
3.7 Navigation..... 54

List of Figures

Figure 1.1: SMBus Block Diagram.....10

List of Tables

No tables in this document.

List of Waveforms

Waveform 1.1: Byte Write (Start + Address)....25
Waveform 1.2: Byte Read....25
Waveform 1.3: Clock Stretching....25
Waveform 1.4: Multi-Master Arbitration....26
Waveform 1.5: Packet Error Check (PEC)....26
Waveform 1.6: Timeout Abort with Bus Recovery....27

1 Smbus Mas Index

Generated: 2026-10-13

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 APB SMBus - Overview

2.1 Overview

The APB SMBus controller provides System Management Bus communication with an APB interface. It is an SMBus 2.0 master: software programs registers over APB, and the core handles the two-wire protocol. All ten transaction types, packet error checking, clock stretching, a bus timeout with recovery, and a strict register decode are implemented and regression-clean as of the GitHub #58 rewrite (2026-09-09).

2.1.1 Key Features

- SMBus 2.0 master
- Quick Command, Send/Receive Byte
- Read/Write Byte/Word
- Block Read/Write and Block Write-Block Read Process Call, up to `FIFO_DEPTH` bytes (parameter, 2..63, default 32)
- PEC (Packet Error Checking), CRC-8 polynomial 0x07, generated on writes and checked on reads
- Programmable clock divider; standard-mode (100 kHz) and fast-mode (400 kHz) timing sets from the same divider, 100.0 kHz and 390.6 kHz at the default divider
- Clock stretching honoured on every release of SCL
- SCL-low timeout with I2C bus recovery; 0 disables the timeout; worst case from a wedged SCL to `busy=0` is five x `SMBUS_TIMEOUT`
- Open-drain signalling: the block only ever drives a 0
- Sticky, W1C interrupt status; a registered interrupt pin
- Strict seventeen-register decode; everything else returns PSLVERR
- Target (slave) mode: address match with an optional general call, an ACK policy, RX and TX paths through the shared FIFOs, clock stretching while software fills the queue, and its own PEC
- Multi-master arbitration: every transmitted bit is read back, and a 1 that reads as 0 releases the bus within the bit

What the block does not do is listed at the end of this chapter under Limitations. The short version is SMBALERT#, Host Notify and ARP; everything else the register map advertises is built.

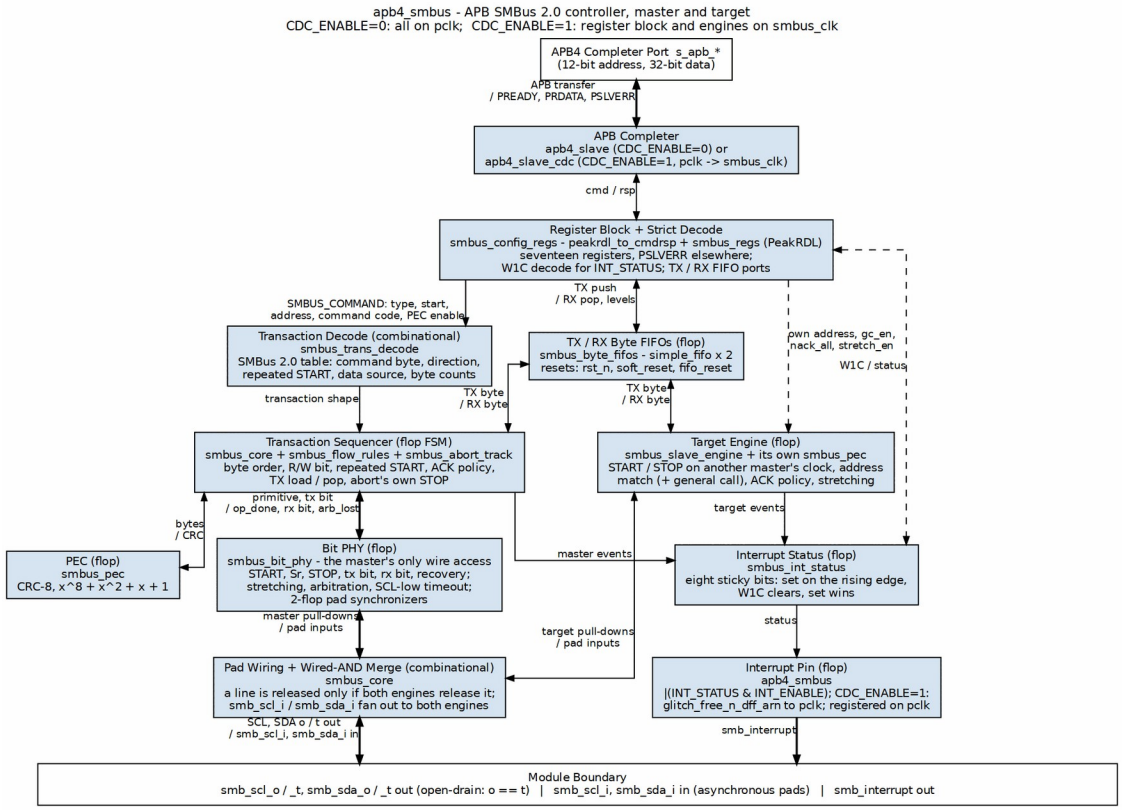
2.1.2 Applications

- Temperature monitoring

- Voltage monitoring
- Fan control
- EEPROM access
- Power management
- System health monitoring

2.1.3 Block Diagram

2.1.3.1 Figure 1.1: SMBus Block Diagram



SMBus Block Diagram

2.2 Architecture

2.2.1 Module decomposition

```
apb4_smbus                  APB4 attach, clocking, the interrupt pin
+- apb4_slave / apb4_slave_cdc   APB4 -> cmd/rsp (CDC_ENABLE picks
which)
+- smbus_config_regs        strict decode, PeakRDL block, W1C decode,
FIFO ports
|  +- peakrdl_to_cmdrsp     cmd/rsp -> PeakRDL passthrough
```

```
|  +- smbus_regs           generated register block
+- smbus_core              TRANSACTION SEQUENCER
   +- smbus_bit_phy        BIT-LEVEL PHY: every SCL and SDA movement
   +- smbus_trans_decode   the transaction table, as combinational
logic
   +- smbus_flow_rules     what happens next: byte states, ACK policy,
the
   |                       TX FIFO load/pop discipline, FIFO
exhaustion
   +- smbus_abort_track    when the abort's OWN STOP has finished
   +- smbus_byte_fifos     TX/RX byte buffers and their three reset
sources
   |  +- simple_fifo x2 -> fifo_sync
   +- smbus_int_status     sticky, edge-set, W1C-cleared interrupt
bits
   +- smbus_pec            CRC-8, polynomial 0x07 (one per engine)
   +- smbus_slave_engine   TARGET ENGINE: the half that answers
      +- smbus_pec         the target's own CRC-8
```

Module	Owns
apb4_smbus	The APB4 attach, the choice of apb4_slave or apb4_slave_cdc, the core clock/reset muxing, and the registered smb_interrupt pin
smbus_config_regs	The strict seventeen-address decode with local PSLVERR, the generated PeakRDL block, the W1C decode for SMBUS_INT_STATUS, and the TX/RX FIFO access ports
smbus_core	The transaction: which bytes go out, in what order, with what R/W bit, where the repeated START goes, which byte is ACKed, what the PEC covers, and when it is done. It owns no bit timing and touches neither SCL nor SDA
smbus_bit_phy	The only module that touches SCL and SDA. Executes one

Module	Owns
	primitive at a time (START, repeated START, STOP, transmit one bit, receive one bit, bus recovery) and reports completion with a single-cycle `op_done`
`smbus_trans_decode`	The SMBus 2.0 transaction table as combinational logic: command byte present, direction, repeated START needed, data source, count byte, data byte count per type
`smbus_flow_rules`	Byte-state classification, byte-complete detection, ACK/NAK policy for received bytes, the TX FIFO load/pop discipline, and TX underrun detection
`smbus_abort_track`	Tells the sequencer when the abort's own STOP has finished, as opposed to the op_done of the primitive being aborted or the PHY's timeout level
`smbus_byte_fifos`	The TX and RX FIFOs (`fifo_sync` via `simple_fifo`) and their three reset sources: `rst_n`, `soft_reset`, `fifo_reset`
`smbus_int_status`	The eight sticky SMBUS_INT_STATUS bits: set on the rising edge of the condition, cleared only by a decoded W1C, set wins over a simultaneous clear
`smbus_slave_engine`	The target: START/STOP detection on somebody else's clock, address match, the ACK policy, the RX and TX byte paths,

Module	Owns
smbus_pec	clock stretching, and its own PEC CRC-8, polynomial x^8 + x^2 + x + 1 (0x07), initial value 0x00

The split that matters is smbus_core / smbus_bit_phy. Quoting the RTL README: “The sequencer owns the transaction and the PHY owns the wire; the sequencer never sees a clock period and the PHY never sees a transaction. They meet at five primitives - START, repeated START, STOP, transmit one bit, receive one bit - each answered with a single-cycle op_done.”

2.2.2 The open-drain contract

SMBus is a wired-AND bus: a device may pull a line LOW or RELEASE it, and may never drive it high. The pins are split smb_scl_i/o/t and smb_sda_i/o/t, where smb_*_o is the value and smb_*_t = 1 means released (input). With that convention the contract collapses to one statement, which is how the PHY implements it:

```
smb_scl_o == smb_scl_t    and    smb_sda_o == smb_sda_t
```

Driving is only ever driving a 0 and a 1 is always a release, so actively driving a 1 on an open-drain line is structurally impossible rather than something a reviewer has to re-check state by state.

2.2.3 Bit timing, fast mode and clock stretching

One SCL period is eight base units. In standard mode the unit is (CLK_DIV + 1) / 2 clocks, so SCL = clk / (4 x (CLK_DIV + 1)); the RDL default of 249 gives 100 kHz from a 100 MHz clock. In fast mode the unit is (CLK_DIV + 1) / 8, so the same divider runs four times faster: SCL = clk / (CLK_DIV + 1), and 249 gives 390.6 kHz. Both unit divides round up, so a small divider cannot silently shorten the period.

SMBUS_CONTROL.fast_mode selects the whole 400 kHz timing set, not just a frequency, and the set is asymmetric. A symmetric period cannot meet fast mode at all: tLOW >= 1.3 us and tHIGH >= 0.6 us do not both fit in two equal halves of a 2.5 us period at the ratio standard mode uses. In fast mode the low phase takes 5/8 of the period and the high phase 3/8.

2.2.3.1 Table 1.1: PHY phase lengths in base units

Phase	Standard	Fast	What it is
LO_A	2	2	SCL low, data driven (tSU;DAT)
HI_A	2	1	SCL released, up to the sample point

Phase	Standard	Fast	What it is
HI_B	2	2	Rest of the high phase
LO_B	2	3	SCL driven low again
HD_STA	4	2	START hold, SDA low to SCL low
SU_STA	4	2	Repeated-START setup, SCL high to SDA low
SU_STO	5	5	STOP setup, SCL high to SDA released
BUF	5	5	Bus free, and the wait before a START

tLOW = LO_B + LO_A and tHIGH = HI_A + HI_B. tSU;STO and tBUF stay at five units in both modes because their fast-mode minimums are not proportionally smaller.

2.2.3.2 Table 1.2: Achieved timing at the default divider

At the RDL default (`CLK_DIV = 249`, 100 MHz core clock), against the SMBus 2.0 / I2C minimums. Every figure is a minimum the block clears, not a target it sits on: the units are integers, so the achieved values sit above the spec rather than on it.

	Standard achieved	Required	Fast achieved	Required
Base unit	125 clk = 1250 ns	-	32 clk = 320 ns	-
SCL period	10.00 us (100.0 kHz)	-	2.56 us (390.6 kHz)	-
tLOW	5000 ns	>= 4700	1600 ns	>= 1300
tHIGH	5000 ns	>= 4000	960 ns	>= 600
tHD;STA	5000 ns	>= 4000	640 ns	>= 600
tSU;STA	5000 ns	>= 4700	640 ns	>= 600

	Standard achieved	Required	Fast achieved	Required
tSU;STO	6250 ns	>= 4000	1600 ns	>= 600
tBUF	6250 ns	>= 4700	1600 ns	>= 1300

A START waits for the bus to be free. Before pulling SDA down the master waits for SCL and SDA to be sampled high together and then holds tBUF. A master that pulls SDA low under a low SCL has not generated a START at all; it has put a data bit on somebody else's transfer. The wait is bounded by `SMBUS_TIMEOUT` only when it is non-zero: a bus that never goes free then reports a timeout rather than hanging. With `SMBUS_TIMEOUT = 0` and a bus that never frees, `busy` stays 1 in the start state until software writes `SMBUS_COMMAND.stop`, which aborts cleanly with a lone STOP.

Clock stretching is honoured on every release of SCL. After letting SCL go, the PHY holds its unit counter at zero until the synchronized SCL input actually reads high, so a slave holding SCL down extends the high phase instead of being clocked through. That applies to data bits, ACK bits, the repeated START and the STOP alike.

2.2.4 Timeout

SCL low for longer than `SMBUS_TIMEOUT` clocks aborts the transaction, reports `SMBUS_STATUS.timeout_error`, releases the bus and returns the FSM to idle. "Low" means low on the bus, either this master pulling it down or the line still reading low after it let go, so it covers a slave stretching forever, a short, and a wedged master.

`SMBUS_TIMEOUT = 0` disables the timeout. It must never mean "expire immediately".

The timeout also covers the START's bus-free wait, and every state that can wait on SCL is timeout-covered, including the final STOP. On expiry the PHY abandons the primitive: it releases both lines and reports. Releasing is the only exit that always exists; a STOP cannot be generated at all while another device is holding SCL down.

A normal bit's low phase is 5/8 of the period, orders of magnitude under any legal SMBus timeout, so the check never fires on healthy traffic. The counter only runs while this master owns a primitive: an idle bus held low by somebody else does not arm it, and the report is consumed once, by the sequencer that acts on it.

Worst case from "SCL wedges" to `busy` = 0 is five timeout windows, about 125 ms at the default `SMBUS_TIMEOUT`. One window for the primitive that stalls, then four for the abort's STOP, which gets `WINDOWS_STOP = 3` (four windows) against `WINDOWS_NORMAL = 0` (one). Cutting an abort off after one window would strand transactions that were a moment from ending; size `SMBUS_TIMEOUT` with the multiplier in mind.

2.2.5 Bus recovery

A STOP needs SDA to rise while SCL is high. If another device is holding SDA down (a slave stuck part-way through a byte, which is what a timeout usually means) the master cannot generate one, and simply releasing leaves every other device watching a transfer that was never terminated.

Whenever the master needs to issue a STOP and finds SDA held low when it goes to release it, and on every timeout abort, it performs I2C bus recovery: with SDA released it pulses SCL up to nine times, open-drain and stretch-aware, sampling SDA at the end of each high phase and stopping as soon as SDA reads high; then it generates a proper STOP (SDA low while SCL is low, SCL high, SDA high). Nine is the I2C limit: it is one more clock than a byte, so a device stuck anywhere inside one has been clocked out of it. A STOP that escalated into recovery and recovered reports `complete=1` with no error: recovery inside a normal STOP is invisible to software.

Recovery clocks are full standard-mode bits, whatever `fast_mode` says: tLOW 5000 ns and tHIGH 5000 ns at the default divider, against the 4.7 us / 4.0 us standard-mode minimums, in both modes. The device being recovered is by definition not keeping up, so the slowest legal clock is the one most likely to shake it loose, and a recovery clock that is only a fraction of a bit low is not standard-mode timing whatever its unit count says. Before the STOP that follows, SCL is driven low a whole phase before SDA moves, never on the same edge.

If SDA is still low after nine clocks the master releases both lines, reports `SMBUS_STATUS.bus_error` (and `timeout_error` as well if a timeout is what started it), and returns to idle without a STOP. `busy` dropping still coincides with both lines released.

Recovery does not run on two exits:

- a timeout in the final STOP itself: the PHY has already released both lines to report it, so re-driving them would be the only way to get it wrong;
- a timeout before anything of ours was framed, unless SDA is the line being held and SCL is quiet. If SCL is also low somebody else is mid-transfer, and pulsing SCL would clock their bits. A stuck SDA on an otherwise quiet bus is recovered even though no START ever went out; a bus found stuck at power-on is exactly what the nine clocks are for.

2.2.5.1 Table 1.4: What the status bits mean together

`complete`	`bus_error`	`timeout_error`	Meaning
1	0	0	the transaction finished and the bus was released with a STOP. This includes a STOP that found SDA held low,

complete	bus_error	timeout_error	Meaning
			escalated into recovery and recovered: recovery inside a normal STOP is invisible to software
0	0	1	a timeout; the bus was released. Recovery either was not applicable (nothing framed, or the STOP itself timed out) or ran and succeeded, and a STOP was generated
0	1	1	a timeout, recovery was attempted and failed: SDA was still low after nine clocks, both lines were released and no STOP reached the wire
0	1	0	aborted without a timeout: `COMMAND.stop` alone, a TX underrun, an RX overrun, or a STOP whose recovery failed
0	x	x	with

complete	bus_error	timeout_error	Meaning
			nak_received or pec_error: the slave did not acknowledge, or the PEC did not match

complete is never set alongside any error. It is computed from the next-state error terms, including the ones raised on the very edge the STOP completes, so a transaction that did not terminate the bus cannot report success.

2.2.5.2 Table 1.3: Measured timeout abort with recovery

Measured against a slave stuck in its ACK after a 15 us stretch with SMBUS_TIMEOUT at 5 us:

Time	Event
0.965 us	START
~5 us	timeout expires; PHY abandons the ACK bit, releases both lines, reports
17.805 us	recovery clock 0 - SCL rises, SDA still low
17.945 us	recovery clock 1 - SCL rises, SDA still low
18.085 us	recovery clock 2 - SCL rises, SDA reads high: the slave let go
18.225 us	SCL high with SDA driven low - the STOP setup
18.345 us	STOP

ending with busy=0, timeout_error=1, bus_error=0 and both lines released. Waveform 1.6 below draws this sequence.

2.2.6 Busy, completion and the abort discipline

SMBUS_STATUS.busy is high from the accepted START request until the FSM is back in idle, whether the transaction succeeded or not, so busy dropping is the single “the bus is yours again” signal.

Every exit runs through a STOP, and error recovery never waits for software. A NAK anywhere (address, command, data or PEC), a PEC mismatch, a FIFO underrun or overrun, or a bus timeout all go to the error state, which issues a STOP, releases both lines and returns to idle on its own. Software does not have to write SMBUS_COMMAND.stop to un-wedge the block.

`busy` dropping to 0 always coincides with both lines released, on every path, including the ones where the STOP itself could not be generated. Every exit is therefore either a real STOP on the wire or a reported release (`bus_error` set, and `timeout_error` if a timeout started it).

An abort (`SMBUS_COMMAND.stop` alone, or a timeout) terminates the primitive that is running, at a point where moving the lines is safe, and then generates the STOP. The abort's first phase pulls SCL low and leaves SDA unchanged, because moving SDA while SCL is high is a START or a STOP, not an abort. `smbus_abort_track` is the single authority on when the sequencer may leave the error state: busy=0 is allowed only when the abort's own STOP has finished, not on the op_done of the primitive being aborted and not on the PHY's timeout level.

`SMBUS_COMMAND.stop` written together with `start` is the ordinary case and is accepted as a no-op, because every transaction ends with a STOP anyway. Written alone while busy it is an abort: the transaction stops, the bus is released and `bus_error` is reported.

`SMBUS_STATUS.complete` is set when a transaction finishes without a PEC mismatch, and never alongside any error (Table 1.4). Read it together with the error bits, not instead of them.

2.2.7 PEC

The CRC-8 (polynomial 0x07) covers every byte that was actually on the wire, in wire order, including both address bytes: the `Addr+W` after the START and the `Addr+R` after the repeated START. It is cleared once, when the START is issued, and never held in clear afterwards.

On a write, the computed PEC is transmitted after the last data byte. On a read, the master ACKs the last data byte instead of NAKing it, receives the slave's PEC byte, NAKs that, and compares it against the running CRC; a mismatch sets `SMBUS_STATUS.pec_error`. The received PEC byte is never folded into the CRC; it is the thing being checked.

`SMBUS_PEC` is written by hardware only at the end of a transaction, so a software-written value survives until a transfer replaces it. After a transfer it holds the transmitted PEC byte on a write and the received PEC byte on a read; the computed CRC is not exposed, so on a PEC error the register shows the slave's byte and `pec_error` says it did not match.

2.2.8 Interrupts

`smb_interrupt` is `(SMBUS_INT_STATUS & SMBUS_INT_ENABLE) != 0`, registered. Not the live status: a pin built from live status cannot be deasserted by software at all, only by the underlying condition going away, which with a level-sensitive interrupt controller upstream is an interrupt storm.

Every `SMBUS_INT_STATUS` bit is sticky: set by the rising edge of its condition and cleared only by a write-1-to-clear, with set winning over a simultaneous clear. That includes the two FIFO threshold bits. Set winning matters because an event arriving in the same cycle software clears the previous one must not be lost: an extra interrupt costs a wasted read, a lost one is a hang.

A NAK counts as an error for `error_int`. It has its own status bit, but it is still a transaction that failed.

The sticky bits live in hardware (`smbus_int_status`); the generated register field is a live mirror of them, and the W1C is decoded in `smbus_config_regs` from `wr_data & wr_biten` at the register's own address, on the rising edge of the held request. The edge detector is armed one cycle after reset so a condition that is true at time zero (an empty TX FIFO) does not set a bit with no access having taken place; SMBUS_INT_STATUS reads 0 out of reset.

2.2.9 Soft reset, FIFO reset and the two-cycle strobe

`SMBUS_CONTROL.soft_reset` is a self-clearing strobe that synchronously restarts the master engine, the bit PHY, the PEC accumulator, both FIFOs and the sticky status, and does not touch the register file. It is a synchronous clear of those blocks, not an asynchronous reset. It always reads back 0. Software keeps everything it programmed and only the engine starts over. The PHY releases both lines as it aborts; stopping mid-primitive while still driving would wedge the bus for every other device.

`SMBUS_CONTROL.fifo_reset` clears the two byte buffers only, leaving the engine alone. It is likewise self-clearing and likewise a synchronous clear.

Every self-clearing strobe is two `pclk` cycles wide as the hardware sees it (`soft_reset`, `fifo_reset`, `COMMAND.start` and `COMMAND.stop`), because the cmd/rsp bridge holds its request for two cycles. Every consumer acts once across that window: `start` is only taken in the idle state, an abort is blocked once the error state has been entered, and the two reset strobes are level clears where a second cycle changes nothing.

2.2.10 Clock domains (CDC_ENABLE)

`CDC_ENABLE` picks the attach, and it is the only thing that changes. With `CDC_ENABLE=0` there is one clock domain: `apb4_slave` converts APB to cmd/rsp and the register block and the whole master engine run on `pclk`/`presetn`; nothing crosses. With `CDC_ENABLE=1`, `pclk` and `smbus_clk` are independent: the register block, `smbus_core` and the PHY run on `smbus_clk`/`smbus_resetn`, and exactly two things cross:

- The APB command and response cross through `apb4_slave_cdc` (`USE_JOHNSON` selects its counter encoding).
- The masked interrupt vector `|(SMBUS_INT_STATUS & SMBUS_INT_ENABLE)`, a level and quasi-static between events, crosses from `smbus_clk` to `pclk` through `glitch_free_n_dff_arn` (three flops), the house level synchronizer, and is then registered on `pclk`. Its destination side takes `presetn`, the reset of the domain it lands in: the source is sticky and re-presents itself after any reset on either side, so no interrupt can be manufactured or lost by the crossing.

SCL and SDA are asynchronous in both builds. They come from pads, and no amount of same-clock design makes a pad synchronous: `smbus_bit_phy` puts two flops on each of them and nothing in the block looks at a raw pin. That is the crossing that always exists; the bus being slow does not make it synchronous.

`CDC_ENABLE` must be 0 or 1; any other value is a `$fatal` at elaboration.

Resets. The block instantiates no reset synchronizer. `presetn` and `smbus_resetn` must be delivered already synchronized by the integrator (asynchronous assert, synchronous deassert), as the other RLB blocks assume.

2.3 Transaction Table

`S` = START, `Sr` = repeated START, `P` = STOP, `A` = master ACKs, `N` = master NAKs. Every byte from the slave is acknowledged by the master as shown; every byte from the master is acknowledged by the slave, and a slave NAK anywhere aborts the transaction. `[PEC]` is present only when `SMBUS_CONTROL.pec_en` is set.

2.3.0.1 Table 1.5: Transaction types and wire sequences

Code	Type	On the wire	Data bytes
0x0	Quick Command	`S, Addr+W, P`	0
0x1	Send Byte	`S, Addr+W, Data, [PEC], P`	1
0x2	Receive Byte	`S, Addr+R, Data(N), [PEC], P`	1
0x3	Write Byte	`S, Addr+W, Cmd, Data, [PEC], P`	1
0x4	Read Byte	`S, Addr+W, Cmd, Sr, Addr+R, Data(N), [PEC], P`	1
0x5	Write Word	`S, Addr+W, Cmd, DataLo, DataHi, [PEC], P`	2
0x6	Read Word	`S, Addr+W, Cmd, Sr, Addr+R, DataLo(A), DataHi(N), [PEC], P`	2
0x7	Block Write	`S, Addr+W, Cmd, Count, Data x Count, [PEC], P`	Count
0x8	Block Read	`S, Addr+W, Cmd, Sr, Addr+R, Count, Data x Count(N), [PEC], P`	Count, from the slave
0x9	Block Proc Call	Block Write then `Sr, Addr+R, Count, Data x Count(N), [PEC], P`	both

With PEC enabled on a read, the last data byte is ACKed rather than NAKed and the slave's PEC byte is the byte that receives the NAK.

Every read that carries a command code has a repeated START. The slave is addressed for WRITE to receive the command byte, then `Sr` and addressed again for READ. Only Receive Byte, which has no command code, addresses for read straight away.

The data byte count is per transaction type. `SMBUS_BLOCK_COUNT` governs the block transfers and nothing else: a Write Byte sends exactly one data byte whatever `BLOCK_COUNT` happens to hold from a previous transfer.

2.3.0.2 Table 1.6: Where the data comes from

Transaction	Transmitted data	Received data
Send Byte, Write Byte	`SMBUS_DATA`	-
Write Word, Block Write	TX FIFO, in order	-
Block Write count byte	`SMBUS_BLOCK_COUNT`	-
Receive Byte, Read Byte	-	RX FIFO and `SMBUS_DATA`
Read Word, Block Read (last byte)	-	`SMBUS_DATA` as well: every received data byte writes it, so it holds the last one
Read Word, Block Read	-	RX FIFO, in order
Block Read count byte	-	from the slave, clamped to the FIFO depth and written back to `SMBUS_BLOCK_COUNT`

A one-byte read lands in `SMBUS_DATA` as well as the FIFO so that reading a single byte needs no FIFO access at all. `SMBUS_DATA` is written by every received data byte, so after a Read Word or Block Read it holds the last byte received; the RX FIFO has them all in order.

2.3.1 Register Summary

Offset	Name	Access	Description
0x00	SMBUS_CONTROL	RW	Global control (enable, mode, PEC, resets)
0x04	SMBUS_STATUS	RO	Status flags and FSM state
0x08	SMBUS_COMMAND	RW	Transaction type, command byte, start/stop
0x0C	SMBUS_SLAVE_ADDR	RW	Target slave address

Offset	Name	Access	Description
0x10	SMBUS_DATA	RW	Single data byte
0x14	SMBUS_TX_FIFO	WO	Transmit FIFO write port
0x18	SMBUS_RX_FIFO	RO	Receive FIFO read port
0x1C	SMBUS_FIFO_STATUS	RO	TX/RX FIFO levels and flags
0x20	SMBUS_CLK_DIV	RW	SCL clock divider
0x24	SMBUS_TIMEOUT	RW	SCL-low limit, 0 = disabled
0x28	SMBUS_OWN_ADDR	RW	The address this target answers, and its enable
0x2C	SMBUS_INT_ENABLE	RW	Interrupt enable mask
0x30	SMBUS_INT_STATUS	W1C	Sticky interrupt status
0x34	SMBUS_PEC	RW	PEC value (CRC-8)
0x38	SMBUS_BLOCK_COUNT	RW	Block transfer byte count
0x3C	SMBUS_SLAVE_CTRL	RW	Target policy: general call, NAK-all, PEC, stretching
0x40	SMBUS_SLAVE_STATUS	RO	Target direction, stretching, PEC verdict and value

Only these seventeen addresses decode. Every other address in the 4 KB window is dropped: no internal strobe fires, the read returns 0, and the access is acknowledged locally with `PSLVERR`. See Chapter 5: Register Map for full field definitions, reset values, the FIFO contract and the programming sequences.

2.4 Parameters

`apb4_smbus` takes three top-level parameters:

Parameter	Default	Description
FIFO_DEPTH	32	TX/RX FIFO depth; the default is the SMBus 2.0 block size. Legal range 2..63 (a $fatal outside it): the level and count fields are six bits, and fifo_sync addresses with $clog2(DEPTH) so a depth of 1 does not elaborate. Every value in the range is lint-clean, with the FIFO level width resized to fit. The slave-supplied block count is clamped to it
CDC_ENABLE	0	0 = single pclk domain (apb4_slave), 1 = independent smbus_clk via apb4_slave_cdc and the interrupt level synchronizer. Only 0 and 1 are legal
USE_JOHNSON	0	Counter encoding forwarded to apb4_slave_cdc

There is no skid-depth parameter.

2.4.1 Integration notes

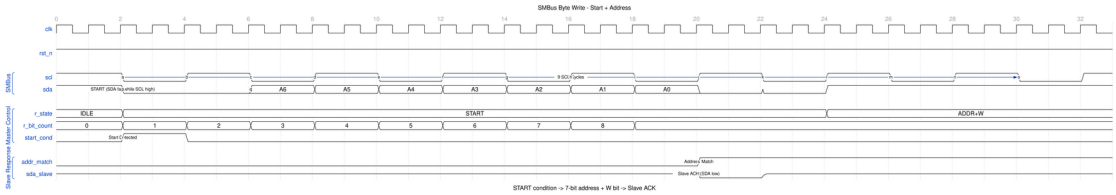
- Deliver presetn (and smbus_resetn when CDC_ENABLE=1) already synchronized: asynchronous assert, synchronous deassert. The block adds no reset synchronizer of its own.
- Wire smb_scl_i/o/t and smb_sda_i/o/t to open-drain pads; the block only ever drives a 0 (smb_*_o == smb_*_t), and it synchronizes both inputs with two flops itself.
- Build with the default active-low reset polarity; see Limitations for the RESET_ACTIVE_HIGH build.

2.5 Waveforms

Waveforms 1.1 to 1.5 draw the protocol elements this controller speaks, in composed wire view; Waveform 1.6 draws the measured timeout abort of Table 1.3. The signal names the captions use are real and are listed in the wavedrom directory README.

2.5.1 Waveform 1.1: Byte Write (Start + Address)

Shows the START condition and 7-bit address transmission.

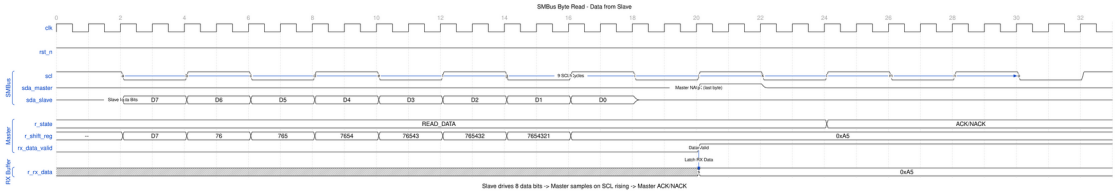


SMBus Byte Write

START condition is SDA falling while SCL is high, after the bus-free wait. The 7-bit slave address plus R/W bit is clocked out, followed by slave ACK (SDA low during the 9th clock).

2.5.2 Waveform 1.2: Byte Read

Shows slave-to-master data transfer.

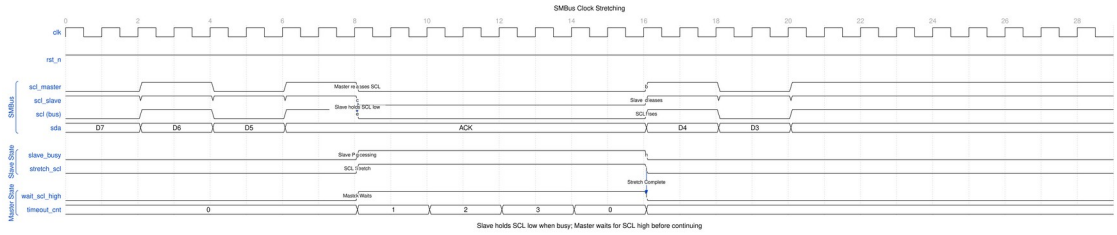


SMBus Byte Read

Slave drives 8 data bits while master clocks SCL. Master samples each bit on SCL rising edge, then provides ACK (more data) or NACK (last byte).

2.5.3 Waveform 1.3: Clock Stretching

Slave flow control by holding SCL low.

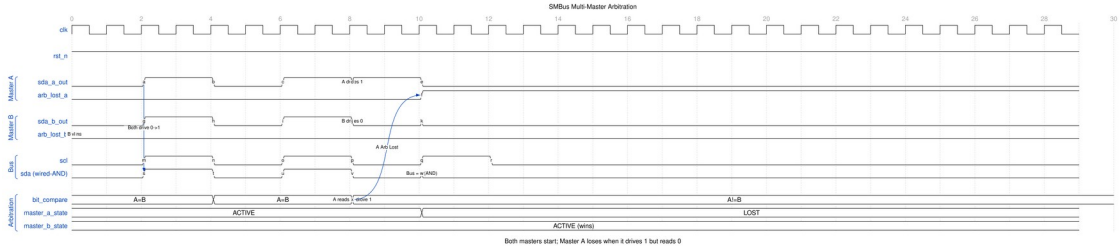


SMBus Clock Stretch

When the slave needs processing time, it holds SCL low after the master releases it. The PHY holds its unit counter at zero until the synchronized SCL input reads high, so the high phase extends and no data is lost. A stretch longer than `SMBUS_TIMEOUT` is a timeout (Waveform 1.6).

2.5.4 Waveform 1.4: Multi-Master Arbitration

Collision detection when multiple masters start simultaneously.



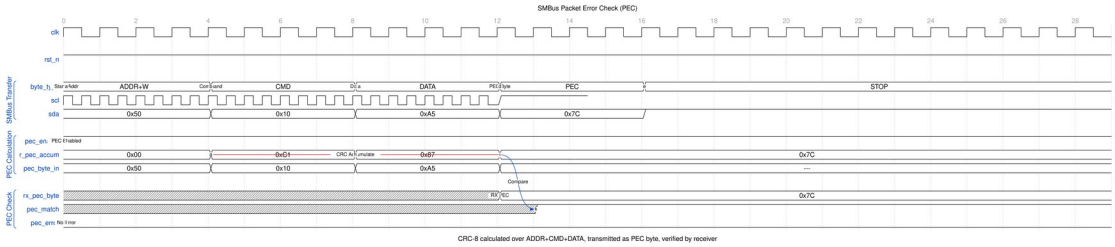
SMBus Arbitration

Every transmitted bit is read back in the SCL-high phase. Sending a 0 means driving SDA down and everyone driving down agrees, so only a **1 that reads back as 0** says another master is still transmitting and has won. START is exempt, because pulling SDA down there is the framing rather than data.

On loss the PHY releases both lines in the same bit and the sequencer reports `SMBUS_STATUS.arb_lost` and returns to idle **without generating a STOP** – the winner's transfer is still in progress, and re-driving the lines to frame a STOP would corrupt it. Retry is left to software, and the bus-free wait before START is what makes the retry safe.

2.5.5 Waveform 1.5: Packet Error Check (PEC)

CRC-8 error detection for data integrity.

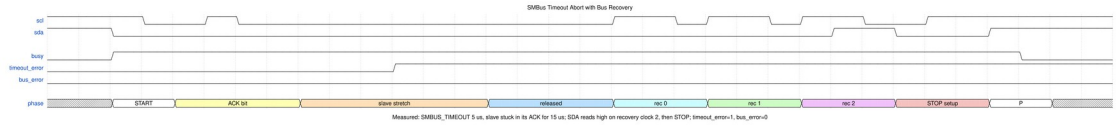


SMBus PEC

PEC is calculated over every byte on the wire, both address bytes included, using CRC-8 polynomial 0x07. On a write the PEC byte is transmitted after the last data byte; on a read it is received, NAKed, and compared.

2.5.6 Waveform 1.6: Timeout Abort with Bus Recovery

The measured sequence of Table 1.3: a slave stuck in its ACK, the timeout expiring, three recovery clocks, and the STOP.



SMBus Timeout Recovery

The PHY releases both lines on expiry, then clocks SCL with SDA released (full standard-mode bits) until SDA reads high at the end of a high phase, then drives SCL low, SDA low a phase later, and generates the STOP. `busy` falls only when that STOP has finished; `timeout_error` is set and `bus_error` is not, because recovery succeeded (Table 1.4, row 2).

2.6 Target mode

The block answers as well as asks. `smbus_slave_engine` is a separate module from `smbus_bit_phy` for a structural reason: **the master owns the clock and a target does not.** Every master primitive is something the PHY schedules; every target action is a response to an edge somebody else produced. One module that is sometimes a clock source and sometimes a passenger would be the wrong shape.

It never drives a 1 either. It emits two pull-down requests and `smbus_core` wired-ANDs them with the master's, which is the rule the bus itself obeys, so the merge needs no ownership mux to be electrically correct. **SDA only ever changes on a falling edge of SCL** – a transition while SCL is high is a START or a STOP, not data – so every drive decision is taken on a fall and held.

control	what it decides
`SMBUS_CONTROL.slave_en`	the engine runs at all
`SMBUS_OWN_ADDR`	the address it answers, and whether it answers one

control	what it decides
SMBUS_SLAVE_CTRL.gc_en	also answer the general call, address 0x00
SMBUS_SLAVE_CTRL.nack_all	software is busy: NAK our own address
SMBUS_SLAVE_CTRL.pec_en	maintain, check and append the target PEC
SMBUS_SLAVE_CTRL.stretch_en	hold SCL while a read waits for software

A full RX FIFO is answered with a NAK rather than a silently dropped byte: the master has to be told, or it is writing into a target that is not listening.

2.6.1 The target PEC never counts bytes

A target does not know how long a transfer is; the protocol does, and the protocol lives in software. CRC-8 removes the need to know. On a **write** the running CRC covers the address byte and every data byte, and a correct trailing PEC byte drives it to **zero**, so “PEC good” is “the running value is zero at the STOP”. On a **read**, when the TX FIFO runs dry the byte sent IS the running CRC, which is exactly the PEC the master is waiting for; software sets the length by how many bytes it queues.

2.6.2 Stretching, or not

With `stretch_en` set, a read that finds the TX FIFO empty holds SCL low until software puts a byte in it, and `SMBUS_SLAVE_STATUS.stretching` says so. With it clear the engine sends `0xFF` instead, which is what an unprogrammed target on a real bus looks like and keeps a slow CPU from wedging the whole wire.

2.6.3 One engine on the wire

A master START is refused while the target half is **answering** – not merely while a transfer is visible on the bus. The distinction matters: SDA pulled low under a high SCL looks exactly like a START that never ends, and a target that claimed the wire for that would block the bus recovery that exists to clear it. Whether the bus is busy is the master PHY’s own question, answered by the bus-free wait in front of every START.

2.7 Limitations

- **A `start` written while `master_en` is clear is silently discarded.** It is not reported as an error, so software that forgets to enable master mode sees a transaction that simply never happens.
- Block Process Call is a block write followed by a repeated START and a block read; the two halves share `SMBUS_BLOCK_COUNT`.

- A slave-supplied block length of 0, or one larger than the FIFO depth, is clamped rather than honoured.
- SMBALERT# and the Host Notify protocol are not implemented; there is no `smbalert_n` pin, and with them goes the Alert Response Address a target would answer.
- ARP, the SMBus address resolution protocol, is not implemented. `SMBUS_OWN_ADDR` is a fixed address software programs.

2.8 Navigation

Next: Chapter 2 (Architecture) is planned and not yet written; the architecture of the block is described above. See the index.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 APB SMBus - Register Map

3.1 Overview

The register map below is generated from `rdl/smbus/smbus_regs.rdl` and matches the decode in `rtl/smbus/smbus_config_regs.sv`. Field descriptions are quoted from the RDL, which is the single source of truth for the fields; the contracts behind them (timing, timeout, FIFOs, interrupts) are from `rtl/smbus/README.md`.

Strict decode. Only the seventeen mapped registers decode. Every other address in the 4 KB window is dropped: no internal strobe fires, the read returns 0, and the access is acknowledged locally with `PSLVERR`. The generated block sees seven address bits, so without the strict decode every unmapped address would alias onto a real register 128 bytes below it (0x080 would write `SMBUS_CONTROL`). It was six bits and 0x040 until the two target registers pushed the top of the map past 0x03F. The drop is acknowledged combinationally and locally because the adapter holds its request until it is acknowledged; a dropped access that is never acknowledged hangs the bus.

Access legend: RW = read/write, RO = read-only, WO = write-only, W1C = write-1-to-clear, AC = self-clearing (hardware clears the bit after the action).

3.1.1 Register Summary

Offset	Name	Access	Reset	Description
0x00	SMBUS_CONTROL	RW	0x00000000	Global control (enable, mode, PEC, resets)
0x04	SMBUS_STATUS	RO	0x00000000	Status flags and FSM state
0x08	SMBUS_COMMAND	RW	0x00000000	Transaction type, command byte, start/stop
0x0C	SMBUS_SLAVE_ADDR	RW	0x00000000	Target slave address (master mode)
0x10	SMBUS_DATA	RW	0x00000000	Single data byte, both directions
0x14	SMBUS_TX_FIFO	WO	0x00000000	Transmit FIFO write port
0x18	SMBUS_RX_FIFO	RO	-	Receive FIFO read port (read pops)
0x1C	SMBUS_F	RO	0x0000808	TX/RX FIFO levels and

Offset	Name	Access	Reset	Description
	IFO_STATUS		0 (live)	flags (post-reset both FIFOs empty: tx_empty bit 7, rx_empty bit 15)
0x20	SMBUS_CLK_DIV	RW	0x000000F9	SCL clock divider
0x24	SMBUS_TIMEOUT	RW	0x002625A0	SCL-low limit in clocks, 0 = disabled; worst case to busy=0 is five windows
0x28	SMBUS_OWN_ADDR	RW	0x00000000	The address this target answers, and its enable
0x2C	SMBUS_INT_ENABLE	RW	0x00000000	Interrupt enable mask
0x30	SMBUS_INT_STATUS	W1C	0x00000000	Sticky interrupt status; reads 0 out of reset
0x34	SMBUS_PEC	RW	0x00000000	PEC value (CRC-8)
0x38	SMBUS_BLOCK_COUNT	RW	0x00000000	Block transfer byte count
0x3C	SMBUS_SLAVE_CTRL	RW	0x00000000	Target policy: general call, NAK-all, PEC, clock stretching
0x40	SMBUS_SLAVE_STATUS	RO	0x00000000	Target direction, stretching, PEC verdict and running value

3.2 Functional Description

Every register, in offset order, with field-level tables. The access legend above applies throughout.

3.2.1 SMBUS_CONTROL (0x00)

Bit	Name	Access	Reset	Description
0	master_en	RW	0	Enable master mode (0=disabled, 1=enabled). `SMBUS_COMMAND.start` is ignored unless this is already set
1	slave_en	RW	0	Enable target mode. With `SMBUS_OWN_ADDR.addr_en` this arms the target engine; see SMBUS_SLAVE_CTRL for its policy
2	pec_en	RW	0	Enable Packet Error Checking (0=disabled, 1=enabled). Sampled when the transaction starts
3	fast_mode	RW	0	0 = 100 kHz standard-mode timing, 1 = 400 kHz fast-mode timing. Selects the whole timing set, not just a frequency (see below)
4	fifo_reset	RW/AC	0	Reset TX/RX FIFOs (write 1, auto-clears). A synchronous clear of the two byte buffers only, leaving the engine alone
5	soft_reset	RW/AC	0	Write 1 to synchronously restart the master engine, the bit PHY, the PEC accumulator and both FIFOs, and to clear the sticky status: a

Bit	Name	Access	Reset	Description
				synchronous clear, not an asynchronous reset. Self-clearing: it always reads back 0; as the hardware sees it the strobe is two `pclk` cycles wide (the bridge holds its request), and every consumer acts once across that window. The register file is not touched
31:6	Reserved	RO	0	Reads 0

fast_mode, from the RDL: “Selects the whole timing SET, not just a frequency, and the set is ASYMMETRIC because a symmetric period cannot meet fast mode at all: tLOW >= 1.3 us and tHIGH >= 0.6 us do not both fit in two equal halves of 2.5 us. The SCL period is eight base units either way; in standard mode the unit is (SMBUS_CLK_DIV+1)/2 and the low and high phases take four units each, in fast mode the unit is (SMBUS_CLK_DIV+1)/8 and the low phase takes five units to the high phase’s three. tHD;STA and tSU;STA shrink from four units to two; tSU;STO and tBUF stay at five units in BOTH modes, because their fast-mode minimums are not proportionally smaller.” The achieved-versus-required table is Table 1.2 in Chapter 1. Bus recovery clocks always use standard-mode timing whatever this bit says.

soft_reset, from the RDL: “The REGISTER FILE IS NOT TOUCHED - software keeps everything it programmed and only the engine starts over. Use it to recover a wedged bus without reconfiguring the block.” The PHY releases both lines as it aborts.

Strobe width. `soft_reset`, `fifo_reset`, `SMBUS_COMMAND.start` and `SMBUS_COMMAND.stop` are all self-clearing strobes that are two `pclk` cycles wide as the hardware sees them, because the cmd/rsp bridge holds its request for two cycles. Every consumer is written to act once across that window: `start` is only taken in the idle state, an abort is blocked once the error state is entered, and the two reset strobes are level clears where a second cycle changes nothing.

3.2.2 SMBUS_STATUS (0x04)

Read-only. Software writes to this register are dropped (no PSLVERR). The error bits and `complete` are result flags: hardware clears all of them when the next `start` is accepted and sets them as the transaction runs, so they hold the outcome of the last transaction until a new one begins. The sticky, W1C event bits are in SMBUS_INT_STATUS (0x30).

Bit	Name	Access	Description
0	busy	RO	Transaction in progress (0=idle, 1=busy). High from the accepted START request until the FSM is back in idle, whether the transaction succeeded or not. Dropping to 0 always coincides with both lines released
1	bus_error	RO	Bus error: a transfer was aborted for a reason other than a NAK or a PEC mismatch - `SMBUS_COMMAND.stop` written alone, a TX FIFO underrun, an RX FIFO overrun, or bus recovery that could not free SDA within nine clocks. Set together with `timeout_error` it means recovery was attempted, failed, and the bus was released without a STOP
2	timeout_error	RO	A bus timeout: SCL was held low - by this master or by anyone else - for longer than SMBUS_TIMEOUT. The transaction was aborted and the bus released. When this bit is set, recovery clocks were issued: every timeout abort runs I2C bus recovery (up to nine SCL pulses with SDA released, standard-mode timing) before its STOP. If SDA was still low after nine clocks, `bus_error` is set as well and no STOP was generated
3	pec_error	RO	PEC mismatch detected: on a

Bit	Name	Access	Description
			read with `pec_en`, the slave's PEC byte differed from the running CRC. `complete` is not set
4	arb_lost	RO	Multi-master arbitration lost: a transmitted 1 read back as 0. Both lines are released within the bit and the sequencer idles WITHOUT framing a STOP, because the winner's transfer is still in progress
5	nak_received	RO	NAK received from slave, at any byte: address, command, data or PEC. A NAK aborts the transaction through a STOP and counts as an error for `error_int`, but does not set `bus_error`
6	slave_addressed	RO	This device is addressed as a target and in a transfer. A LEVEL, true until the STOP; the sticky version is `SMBUS_INT_STATUS.slave_addr_int`
7	complete	RO	Transaction completed successfully: the transaction finished and the bus was released with a STOP. Never set alongside any error; it is computed from the next-state error terms, including the ones raised on the very edge the STOP completes, so a transaction that did not terminate the bus cannot report success

Bit	Name	Access	Description
11:8	fsm_state	RO	Current state machine state (for debugging); the encoding is ABI, see below
31:12	Reserved	RO	Reads 0

3.2.2.1 What the status bits mean together

`complete`	`bus_error`	`timeout_error`	Meaning
1	0	0	the transaction finished and the bus was released with a STOP. This includes a STOP that found SDA held low, escalated into recovery and recovered: recovery inside a normal STOP is invisible to software
0	0	1	a timeout; the bus was released. Recovery either was not applicable (nothing framed, or the STOP itself timed out) or ran and succeeded, and a STOP was generated
0	1	1	a timeout, recovery was attempted and failed: SDA was

complete	bus_error	timeout_error	Meaning
			still low after nine clocks, both lines were released and no STOP reached the wire
0	1	0	aborted without a timeout: `COMMAND.stop` alone, a TX underrun, an RX overrun, or a STOP whose recovery failed
0	x	x	with `nak_received` or `pec_error`: the slave did not acknowledge, or the PEC did not match

Recovery does not run on two timeout exits: a timeout in the final STOP itself (the PHY has already released both lines to report it), and a timeout before anything of ours was framed unless SDA is the line being held and SCL is quiet (if SCL is also low somebody else is mid-transfer, and pulsing SCL would clock their bits). A stuck SDA on an otherwise quiet bus is recovered even though no START went out. Recovery clocks are full standard-mode bits (tLOW 5000 ns, tHIGH 5000 ns at the default divider) in both modes.

3.2.2.2 fsm_state encoding

Value	State	Value	State
0x0	IDLE	0x8	DATA_RD
0x1	START	0x9	DATA_RD_ACK
0x2	ADDR	0xA	PEC_WR
0x3	ADDR_ACK	0xB	PEC_WR_ACK
0x4	CMD	0xC	PEC_RD
0x5	CMD_ACK	0xD	STOP

Value	State	Value	State
0x6	DATA_WR	0xE	ERROR
0x7	DATA_WR_ACK	0xF	RESTART

ERROR is the state every abort passes through; it issues the STOP (bus recovery first if SDA is stuck), releases both lines and returns to IDLE on its own. Software never has to act to leave it.

3.2.3 SMBUS_COMMAND (0x08)

Bit	Name	Access	Reset	Description
3:0	trans_type	RW	0	Transaction type (see Transaction Types)
7:4	Reserved	RO	0	Reads 0
15:8	cmd_code	RW	0	SMBus command byte (for transactions that use it)
16	start	RW/AC	0	Write 1 to launch the transaction described by the rest of this register. Self-clearing. It is ignored unless `SMBUS_CONTROL.master_en` is already set - the write is accepted and silently discarded, `SMBUS_STATUS.busy` never rises and no error is reported, so software must enable master mode before starting
17	stop	RW/AC	0	Written together with `start` it is a no-op: every transaction ends with a STOP anyway. Written alone while `SMBUS_STATUS.busy` is set

Bit	Name	Access	Reset	Description
				it is an abort - the transaction stops, a real STOP is generated on the wire, both lines are released, `busy` drops and `SMBUS_STATUS.bus_error` is set. Self-clearing
31:18	Reserved	RO	0	Reads 0

`trans_type`, `cmd_code`, `SMBUS_SLAVE_ADDR`, `SMBUS_CONTROL.pec_en` and the transaction's data byte count are sampled when `start` is accepted; changing them while `busy` is set does not affect the running transaction.

3.2.3.1 Transaction Types

`trans_type` is a 4-bit field. Block Write and Block Read are separate encodings. `[PEC]` is present only when `pec_en` is set; `(N)` marks the byte the master NAKs.

trans_type	Description	On the wire	Data bytes
0	Quick Command	`S, Addr+W, P`	0
1	Send Byte	`S, Addr+W, Data, [PEC], P`	1
2	Receive Byte	`S, Addr+R, Data(N), [PEC], P`	1
3	Write Byte	`S, Addr+W, Cmd, Data, [PEC], P`	1
4	Read Byte	`S, Addr+W, Cmd, Sr, Addr+R, Data(N), [PEC], P`	1
5	Write Word	`S, Addr+W, Cmd, DataLo, DataHi, [PEC], P`	2
6	Read Word	`S, Addr+W, Cmd, Sr, Addr+R, DataLo(A), DataHi(N), [PEC], P`	2
7	Block Write	`S, Addr+W, Cmd, Count, Data x Count, [PEC], P`	BLOCK_COUNT

trans_type	Description	On the wire	Data bytes
8	Block Read	`S, Addr+W, Cmd, Sr, Addr+R, Count, Data x Count(N), [PEC], P`	Count, from the slave
9	Block Write-Block Read Process Call	Block Write then `Sr, Addr+R, Count, Data x Count(N), [PEC], P`	both

Every read that carries a command code has a repeated START. The data byte count is per transaction type: `SMBUS_BLOCK_COUNT` governs the block transfers and nothing else. Values 10 to 15 are not transaction types; they decode with no command byte, no data and write direction, so they go out as a Quick Command.

3.2.4 SMBUS_SLAVE_ADDR (0x0C)

Bit	Name	Access	Reset	Description
6:0	slave_addr	RW	0	Target slave address, 7 bits. The R/W direction is not in this field - it comes from `SMBUS_COMMAND.trans_type`, and the controller appends it when it puts the address byte on the wire (twice, for a read with a command code)
31:7	Reserved	RO	0	Reads 0

There is no writable R/W bit at bit 7; writes to bit 7 are ignored and it reads back 0. Quick Command has both directions, and each is its own transaction type rather than a direction bit: type 0x0 sends the address with R/W = 0 and type 0xA with R/W = 1. The R/W bit IS the payload of a quick command, so a shared direction bit would mean nothing for the other nine types.

3.2.5 SMBUS_DATA (0x10)

Bit	Name	Access	Reset	Description
7:0	data	RW	0	Data byte. Software writes it for Send Byte / Write Byte; hardware writes it with every received data byte, so after a Receive Byte / Read Byte it holds that byte and after a Read Word / Block Read it holds the last byte received (the RX FIFO has them in order). The hardware path is qualified with `we` so it only lands when there is a received byte
31:8	Reserved	RO	0	Reads 0

A software-written byte therefore survives until a read replaces it; a write-then-read returns what software wrote. A one-byte read lands here as well as in the RX FIFO, so reading a single byte needs no FIFO access at all.

3.2.6 SMBUS_TX_FIFO (0x14)

Write-only port into the transmit FIFO (`FIFO_DEPTH` bytes, 32 by default). Each write pushes one byte. Reads return 0.

Bit	Name	Access	Reset	Description
7:0	tx_data	WO	0	Write data byte to TX FIFO
31:8	Reserved	RO	0	Reads 0

The push takes its byte from the write data of the access that causes it, on a single-cycle edge, so back-to-back writes each push exactly once and the byte pushed is the byte written. The byte enable is part of the decode: a write to this register with the data byte lane disabled (`PSTRB[0]=0`)

is not a FIFO push at all, rather than a push of 0x00. A write while the FIFO is full is dropped by `fifo_sync`; check `SMBUS_FIFO_STATUS.tx_full` first.

3.2.7 SMBUS_RX_FIFO (0x18)

Read-only port from the receive FIFO (`FIFO_DEPTH` bytes, 32 by default). Each read pops one byte. A read with the FIFO empty returns the stale head byte, not 0; check `SMBUS_FIFO_STATUS.rx_empty` (or the level) first.

Bit	Name	Access	Description
7:0	rx_data	RO	Read data byte from RX FIFO
31:8	Reserved	RO	Reads 0

3.2.8 SMBUS_FIFO_STATUS (0x1C)

Read-only FIFO level and flag register.

Bit	Name	Access	Description
5:0	tx_level	RO	Number of bytes in TX FIFO (0..`FIFO_DEPTH`, 0-32 at the default depth)
6	tx_full	RO	TX FIFO full flag
7	tx_empty	RO	TX FIFO empty flag
13:8	rx_level	RO	Number of bytes in RX FIFO (0..`FIFO_DEPTH`, 0-32 at the default depth)
14	rx_full	RO	RX FIFO full flag
15	rx_empty	RO	RX FIFO empty flag
31:16	Reserved	RO	Reads 0

3.2.9 SMBUS_CLK_DIV (0x20)

Bit	Name	Access	Reset	Description
15:0	clk_div	RW	0x00F9 (249)	SCL clock divider

Bit	Name	Access	Reset	Description
31:16	Reserved	RO	0	Reads 0

From the RDL: “STANDARD MODE: SCL = sys_clk / (4 * (div + 1)) - the default 249 gives 100 kHz at 100 MHz. FAST MODE (SMBUS_CONTROL.fast_mode=1) is four times faster from the SAME divider: SCL = sys_clk / ((div + 1)), i.e. 8 units of (div+1)/8 clocks - 249 gives 390.6 kHz at 100 MHz. Both unit divides round UP, so a small divider cannot silently shorten the period.”

3.2.10 SMBUS_TIMEOUT (0x24)

Bit	Name	Access	Reset	Description
23:0	timeout	RW	0x2625A0 (2,500,000)	Bus timeout in core clock cycles. It measures how long SCL is low on the bus - either this master pulling it down or the line still reading low after we let go - so it catches a stretching slave, a short and a wedged master alike. It is counted only while this master owns a primitive; an idle bus held low by somebody else does not arm it. 0 disables the check entirely; it must never mean “expire immediately”. Default 2500000 is ~25 ms at 100 MHz
31:24	Reserved	RO	0	Reads 0

Expiry aborts the transaction, reports `SMBUS_STATUS.timeout_error`, runs bus recovery (with the two exceptions under SMBUS_STATUS), releases the bus and returns the FSM to idle. The window also bounds the bus-free wait before a START, and every state that can wait on SCL is covered, the final STOP included. With the timeout disabled (0) the bus-free wait is unbounded: on

a bus that never frees, `busy` stays 1 in the start state until software writes `SMBUS_COMMAND.stop`, which aborts cleanly with a lone STOP.

Worst case from "SCL wedges" to `busy=0` is five times this value (about 125 ms at the default): one window for the primitive that stalls, then four for the abort's STOP, which is deliberately more patient. Size the value with the multiplier in mind.

3.2.11 SMBUS_OWN_ADDR (0x28)

Bit	Name	Access	Reset	Description
6:0	own_addr	RW	0	7-bit own slave address
7	addr_en	RW	0	Enable own address matching (slave mode)
31:8	Reserved	RO	0	Reads 0

`addr_en` is what makes the target answer an address at all: with it clear the compare is disabled and the engine responds to nothing, whatever `SMBUS_CONTROL.slave_en` says.

3.2.12 SMBUS_INT_ENABLE (0x2C)

Bit	Name	Access	Reset	Description
0	complete_en	RW	0	Enable interrupt on transaction complete
1	error_en	RW	0	Enable interrupt on bus error (any of `bus_error`, `timeout_error`, `pec_error`, `nak_received`)
2	tx_thresh_en	RW	0	Enable the TX threshold interrupt. The threshold is fixed at empty: the condition is `tx_fifo_empty`, not a

Bit	Name	Access	Reset	Description
				programmable level
3	rx_thresh_en	RW	0	Enable the RX threshold interrupt. The threshold is fixed at non-empty: the condition is `!rx_fifo_empty`, not a programmable level
4	slave_addr_en	RW	0	Enable interrupt when addressed as a target
5	slave_rx_en	RW	0	Enable interrupt when the target takes a byte off the bus into the RX FIFO
6	slave_tx_en	RW	0	Enable interrupt when the target needs a byte to transmit and the TX FIFO is empty. With `SMBUS_SLAVE_CTRL.stretch_en` set the bus is being held while this is true, so software is on the clock
7	slave_done_en	RW	0	Enable interrupt when a transfer addressed to this target ends at the STOP. `SMBUS_SLAVE_STATUS.pec_error` is valid from that moment
31:8	Reserved	RO	0	Reads 0

There is no programmable threshold register; the two threshold bits are edge-set on the fixed conditions above.

3.2.13 SMBUS_INT_STATUS (0x30)

From the RDL: “Interrupt status flags (write 1 to clear). Every bit is sticky: set by the RISING EDGE of its condition and cleared only by a W1C write, with SET WINNING over a simultaneous clear. The smb_interrupt pin is (SMBUS_INT_STATUS & SMBUS_INT_ENABLE) != 0, registered - so clearing a bit here is what deasserts the pin.”

Bit	Name	Access	Reset	Description
0	complete_int	W1C	0	Transaction completed: rising edge of `SMBUS_STATUS.complete`
1	error_int	W1C	0	Bus error occurred: rising edge of any of `bus_error`, `timeout_error`, `pec_error`, `nak_received`
2	tx_thresh_int	W1C	0	TX FIFO became empty (the threshold is fixed at empty). Sticky: set on the edge of the condition, cleared only by writing 1. It is not a live level
3	rx_thresh_int	W1C	0	RX FIFO became non-empty (the threshold is fixed at non-empty). Sticky: set on the edge of the condition, cleared only by writing 1
4	slave_addr_int	W1C	0	Addressed as a target: rising edge of `SMBUS_STATUS.slave_addressed`
5	slave_rx_int	W1C	0	The target took a byte off the bus into the RX FIFO
6	slave_tx_int	W1C	0	The target needs a byte to transmit and the TX FIFO is empty
7	slave_don	W1C	0	A transfer addressed to

Bit	Name	Access	Reset	Description
	e_int			this target ended at the STOP
31:8	Reserved	RO	0	Reads 0

The sticky bits live in hardware (`smbus_int_status`); the generated field is a live mirror of them. The W1C is decoded in `smbus_config_regs` as `wr_data & wr_biten` at this register's address, on the rising edge of the held request: a byte-strobed write clears only bits in the bytes it wrote, and the two-cycle request clears once. Writing 0 to a bit leaves it alone.

The register reads 0 out of reset: the edge detector is armed one cycle after reset so a condition that is already true at time zero (an empty TX FIFO) does not set a bit with no access having taken place.

`smb_interrupt` is one register stage behind the masked status (`CDC_ENABLE=0`), or the masked status through a three-flop level synchronizer onto `pclk` plus one register stage (`CDC_ENABLE=1`). `soft_reset` clears the sticky status.

3.2.14 SMBUS_PEC (0x34)

Bit	Name	Access	Reset	Description
7:0	pec	RW	0	PEC value (CRC-8, polynomial 0x07). Hardware writes the computed PEC of the transaction that just finished, and only then (we), so a software-written value survives until a transaction replaces it. After a transfer it holds the transmitted PEC byte (writes) or the received PEC byte (reads); the computed CRC is not exposed, so on a PEC error the register shows the slave's byte and the comparison result is

Bit	Name	Access	Reset	Description
				SMBUS_STATUS.pec_error
31:8	Reserved	RO	0	Reads 0

The CRC covers every byte that was actually on the wire, in wire order, including both address bytes. The received PEC byte is never folded into the CRC; it is the thing being checked. Software does not need to program this register for a transaction; hardware computes the PEC itself.

3.2.15 SMBUS_BLOCK_COUNT (0x38)

Bit	Name	Access	Reset	Description
5:0	block_count	RW	0	Block transfer byte count (1-32). Software programs it for Block Write; for Block Read the count comes from the slave, and hardware writes it back here (we) clamped both ways: a count of 0 becomes 1, and a count longer than the FIFO depth becomes the depth, so software knows how many bytes to drain. It governs the block transfers only - a Write Byte sends one data byte whatever this holds
31:6	Reserved	RO	0	Reads 0

For a Block Write the count byte on the wire is this value and the payload is `block_count` bytes from the TX FIFO. The SMBus-legal range is 1-32; the master-side clamp sends a programmed 0 as 1 and anything above `FIFO_DEPTH` as `FIFO_DEPTH`, while the register reads back what software wrote. A count larger than the number of staged bytes is a TX underrun (see The FIFO Contract). For a Block Read, a slave-supplied count of 0 is clamped to 1 and one larger than `FIFO_DEPTH` is clamped to the depth, and the clamped value is what lands here. Block Process Call shares this register between its write and read halves.

3.2.16 SMBUS_SLAVE_CTRL (0x3C)

Target-mode policy. `SMBUS_CONTROL.slave_en` turns the engine on and `SMBUS_OWN_ADDR` says which address it answers; these are the choices that have no obvious right answer.

Bit	Name	Access	Reset	Description
0	gc_en	RW	0	Answer the general call address 0x00 as well as the own address. The general call is a write by definition, so a read to 0x00 is not a match
1	nack_all	RW	0	Software is busy: NAK this target's own address instead of answering it. A NAK is the only way a target can say "not now" without holding the bus, which is what the alternative – stretching the clock until software catches up – does to every other device on the wire
2	pec_en	RW	0	Maintain a CRC-8 over every byte of a transfer addressed to this target, the address bytes included. See the note below
3	stretch_en	RW	0	Hold SCL low while a read waits for software to put a byte in the TX FIFO. With this clear an empty FIFO sends 0xFF instead, which is what an unprogrammed target

Bit	Name	Access	Reset	Description
				on a real bus looks like and keeps a slow CPU from wedging the whole bus
31:4	Reserved	RO	0	Reads 0

The target PEC never counts bytes. A target does not know how long a transfer is; the protocol does, and the protocol lives in software. CRC-8 removes the need to know. On a **write** the running value covers the address byte and every data byte, and a correct trailing PEC byte drives it to **zero**, so “PEC good” is “the running value is zero at the STOP”. On a **read**, when the TX FIFO runs dry the byte sent IS the running CRC, which is exactly the PEC the master is waiting for; software sets the length by how many bytes it queues.

3.2.17 SMBUS_SLAVE_STATUS (0x40)

What the target engine is doing. Read-only: every bit is a live view of the engine, and the sticky versions software can poll at leisure are in SMBUS_INT_STATUS.

Bit	Name	Access	Reset	Description
0	rd_not_wr	RO	0	Direction of the transfer in progress: 1 = the master is reading from this target, 0 = writing to it
1	stretching	RO	0	The engine is holding SCL low, waiting for software to put a byte in the TX FIFO
2	pec_error	RO	0	The last WRITE transfer addressed to this target ended with a running CRC that was not zero, which means the master’s PEC byte did not match. Valid from the STOP; cleared at the next START

Bit	Name	Access	Reset	Description
7:3	Reserved	RO	0	Reads 0
15:8	pec_value	RO	0	The running CRC-8. Zero after a correct write PEC; the byte that will be sent when a read runs out of queued data
31:16	Reserved	RO	0	Reads 0

3.3 The FIFO Contract

Load and pop. A transmitted byte is loaded from the TX FIFO and popped in the same cycle; the FIFO's read data is its combinational head, so the value captured on the edge that pops is the byte being loaded. Write Word and Block Write take their data from the TX FIFO in order; Send Byte and Write Byte take it from `SMBUS_DATA` and generate no FIFO traffic; a Block Write's first transmitted byte after the command is the count.

TX underrun (`SMBUS_BLOCK_COUNT` larger than the number of staged bytes): after the last staged byte the master stops, sets `error_int`, and fabricates nothing. A byte nobody asked to send is worse on a shared bus than a short transfer. `SMBUS_STATUS.bus_error` is set; the abort runs through the error state and a STOP.

RX overrun (a block read longer than the free space in the RX FIFO): the master checks `rx_fifo_full` before storing each data byte, the last one included. The first byte that does not fit is not stored, is NAKed, and ends the transfer with `bus_error` and `error_int`; the RX FIFO holds exactly what fit and `complete` stays 0. Checking after the store would be useless: a full `fifo_sync` drops the write silently.

Block-count clamp. A slave-supplied block count of 0, or one larger than the FIFO depth, is clamped to 1 and to the depth respectively. The over-length test looks at the whole byte: a count of 0x40 has all-zero low six bits, so testing a truncated field first would read 64 as 0.

Three things empty the FIFOs, and all three mean the same thing to both: `presetn`/`smbus_resetn`, `SMBUS_CONTROL.soft_reset`, and `SMBUS_CONTROL.fifo_reset`. The two strobes clear both FIFOs over the strobe window, and nothing is accepted into them during that window. A `fifo_reset` landing during an active receive discards the bytes received so far, including one landing in the clear window; the transfer itself still completes or aborts normally.

Levels and flags are consistent at every cycle by construction: `tx_level`/`rx_level` and the empty/full flags are derived from the same counter, so software never sees a level that disagrees with its flag. The level range is 0..`FIFO_DEPTH`, not 0..32.

An `SMBUS_RX_FIFO` read with the FIFO empty returns the stale head byte, not 0. Read the level or `rx_empty` before popping.

3.4 Programming Sequences

Common setup, once:

1. SMBUS_CLK_DIV and SMBUS_TIMEOUT (0 disables the timeout).
2. SMBUS_CONTROL: set master_en, and pec_en / fast_mode if wanted. start is ignored unless master_en is already set, so this write must precede the command.
3. SMBUS_INT_ENABLE if interrupts are wanted; otherwise poll.

Per transaction, then wait for SMBUS_STATUS.busy to fall (or complete_int / error_int) and read the error bits together with complete:

Transaction	Before start	SMBUS_COMMAND	After busy falls
Quick Command (0)	SMBUS_SLAVE_ADDR	trans_type=0, start	check nak_received
Send Byte (1)	SMBUS_SLAVE_ADDR, SMBUS_DATA	trans_type=1, start	-
Receive Byte (2)	SMBUS_SLAVE_ADDR	trans_type=2, start	read SMBUS_DATA (or pop SMBUS_RX_FIFO)
Write Byte (3)	SMBUS_SLAVE_ADDR, SMBUS_DATA	trans_type=3, cmd_code, start	-
Read Byte (4)	SMBUS_SLAVE_ADDR	trans_type=4, cmd_code, start	read SMBUS_DATA (or pop SMBUS_RX_FIFO)
Write Word (5)	SMBUS_SLAVE_ADDR; push low byte then high byte to SMBUS_TX_FIFO	trans_type=5, cmd_code, start	-
Read Word (6)	SMBUS_SLAVE_ADDR	trans_type=6, cmd_code, start	pop SMBUS_RX_FIFO twice: low byte, then high byte
Block Write (7)	SMBUS_SLAVE_ADDR; push N bytes to SMBUS_TX_FIFO; SMBUS_BLOCK_COUNT=N	trans_type=7, cmd_code, start	-
Block Read (8)	SMBUS_SLAVE_ADD	trans_type=8,	read

Transaction	Before `start`	`SMBUS_COMMAND`	After `busy` falls
	R	`cmd_code`, `start`	`SMBUS_BLOCK_COUNT` for the (clamped) count, then pop that many bytes from `SMBUS_RX_FIFO`
Block Process Call (9)	as Block Write	`trans_type=9`, `cmd_code`, `start`	as Block Read; `SMBUS_BLOCK_COUNT` now holds the slave's count

With `pec_en` set, hardware appends the PEC on writes and checks it on reads; `SMBUS_PEC` holds the transmitted or compared value afterwards and `pec_error` reports a mismatch. Nothing extra is programmed.

Recovering from an error. Nothing is required: every error path issues a STOP (bus recovery first if SDA is stuck), releases both lines and returns to idle by itself. Read the error bits together (the truth table under SMBUS_STATUS says whether a STOP reached the wire), drain or `fifo_reset` the FIFOs as needed, and start the next transaction. Allow up to five `SMBUS_TIMEOUT` windows for `busy` to fall after a wedged SCL. `SMBUS_COMMAND.stop` written alone aborts a transaction that software no longer wants; `soft_reset` restarts the engine without touching the register file if something else has gone wrong.

Interrupt service. Read `SMBUS_INT_STATUS`, act, then write 1 to the bits handled. The pin deasserts (one or a few `pclk` later, per `CDC_ENABLE`) once no enabled bit remains set; an event that arrives in the same cycle as the clear is kept.

3.5 Remaining limitations

- A `start` written while `master_en` is clear is silently discarded and not reported.
- Block Process Call shares `SMBUS_BLOCK_COUNT` between its two halves.
- SMBALERT# and Host Notify are not implemented, and with them the Alert Response Address a target would answer.
- ARP is not implemented; `SMBUS_OWN_ADDR` is a fixed address software programs.
- No reset synchronizer is instantiated; `presetn` / `smbus_resetn` must arrive already synchronized (asynchronous assert, synchronous deassert).

3.6 History

The byte engine (command, data and PEC bytes after the address; the repeated START on every read with a command code), SCL generation reaching the pin open-drain, clock stretching, the SCL-low timeout with 0 = disabled and bus recovery, the sticky W1C interrupt status and the registered interrupt pin, the hardware-write-on-result `SMBUS_DATA` / `SMBUS_PEC` / `SMBUS_BLOCK_COUNT` fields, the TX FIFO push from write data, per-type data byte counts, PEC generation and checking, `soft_reset` and `fast_mode`, and the strict decode were all fixed on 2026-09-09 under GitHub issue #58.

3.7 Navigation

Back to: SMBus Specification Index